

YavOracle: Coverage-Safe Fiat–Shamir and Fischlin in Rust

By Yavor Litchev

Motivation: Omission Bugs in FS Transcripts

- ❖ FS: simple in theory, error-prone in code
- ❖ If a prover-controlled, verification-relevant value is not absorbed into the hash state, the implemented transcript no longer matches the proven security model.
- ❖ Such omissions can permit forged proofs, even when the underlying group, hash, and protocol are cryptographically sound.
- ❖ Seen in Merlin, Plonk variants, Solana, etc.
- ❖ What we need is coverage of all verification-relevant data

```
fs.absorb("commitment", &t);  
let e = fs.challenge("e");  
// BUG: forgot to absorb response z  
fs.absorb("response", &z);
```

Coverage Safety: From Intuition to a Property

- ❖ Focus on prover-controlled, verification-relevant values V_P
- ❖ Obligations $O=\{o_1,\dots,o_k\}$ linked to (v_j,ℓ_j)
- ❖ Before any `challenge`, all obligations discharged
- ❖ Coverage-safe transcript condition:

$$(S_0, \sigma_0) \xrightarrow{\tau'} (S', \sigma') \Rightarrow S' = \emptyset$$

for every prefix $\tau' \cdot \text{challenge}(\ell, c)$ with initial $S_0=O$

Architecture: Three Layers for Coverage-Safe Transcripts

- ❖ Message level: `#[derive(FsrBindable)]`
- ❖ Transcript level: `Transcript<PENDING, 0>` typestate
- ❖ Proof level: `prove!` / `verify!` DSL, FS/Fischlin-agnostic

```
annotated structs
  → OBLIG_MASK bitmasks
    → Transcript<PENDING, 0>
      → prove!/verify! over FS / Fischlin / interactive
```

Message DSL: Annotated Structs -> Bindable Obligations

- ❖ Messages as Rust structs
- ❖ Field attributes drive coverage:
 - `#[bind(ob = N)]` – obligatory, pre-challenge
 - `#[bind(skip)]` – deliberately not bound
 - `#[bind(label = "...")]` – domain separation
- ❖ Derive macro emits `Bindable` + `OBLIG_MASK`

```
#[derive(Clone, Debug, FsrBindable)]  
#[bind(prefix = "Commit")]  
pub struct Commit {  
    #[bind(ob = 0)]  
    pub T: G1,  
}
```

```
impl Bindable for Commit {  
    const OBLIG_MASK: u128 = 1 << 0;  
    fn bind<O: Absorb>(&self, o: &mut O, prefix: &str) {  
        o.absorb("Commit.T", &self.T);  
    }  
}
```

Typed Transcripts and Swap-in Oracles

- ❖ `Transcript<const PENDING: u128, 0>` carries pending obligations
- ❖ `absorb::<MASK, M>` clears bits: $P \rightarrow P \& \text{!MASK}$
- ❖ `challenge()` only for `Transcript<0, 0>`
- ❖ Same API for interactive, FS, and Fischlin oracles

```
pub struct Transcript<const PENDING: u128, 0> {
    oracle: 0,
}

impl<const P: u128, 0> Transcript<P, 0> {
    pub fn absorb<const MASK: u128, M: Bindable>(
        self, label: &str, m: &M,
    ) -> Transcript<{ P & !MASK }, 0> { /* ... */ }
}

impl<0> Transcript<0, 0> {
    pub fn challenge<T: FromChallenge>(
        self, label: &str,
    ) -> (T, Transcript<0, 0>) { /* ... */ }
}
```

Proof DSL: prove! / verify! Across Transforms

- ❖ `transform = "fs" | "fischlin"` selects runtime
- ❖ `require = ["c_0", "c_1", ...]` declares label-level coverage
- ❖ `bind` closure replays labeled absorbs
- ❖ Static check: every required label is absorbed

```
prove! {  
  transform = "fs",  
  require = ["c_0", "c_1"],  
  bind = |o: &mut _, i: usize, m_bytes: &[u8]| {  
    let (t0b, t1b) = m_bytes.split_at(8);  
    fsr_core::TranscriptRuntime::absorb(o, "c_0", t0b);  
    fsr_core::TranscriptRuntime::absorb(o, "c_1", t1b);  
  },  
  /* ... */  
}
```

Evaluation: FS/Fischlin Interchange and Omission Bug Detection

- ❖ Case studies: Schnorr, Σ -OR, Σ -AND, multi-round sub-protocols
- ❖ Same protocol under FS and Fischlin via **transform** flag
- ❖ FS vs. Fischlin proof size:

```
FS ok = true, Fischlin ok = true
FS proof bytes: 38
Fischlin proof bytes: 504
```

- ❖ Curated omission bugs rejected at compile time

```
error[fsv-coverage]: missing absorb() for required labels before challenge
--> examples/sigma_or_dsl_fs_bug.rs:42:5
|
42 | prove! {
|   ^^^^^^ missing required label: "c_0"
|
= help: add
      fsr_core::TranscriptRuntime::absorb(o, "c_0", <bytes>);
to the bind closure for this proof
```

Limitations and Future Work

- ❖ Coverage relative to annotations & spec
- ❖ Rust-specific: uses nightly `generic_const_exprs`
- ❖ Does not address circuit-level / algebraic flaws
- ❖ In the future, we would pursue integration and extensions

```
// Example round alias:  
declare_round!(R1 = [Commit]);  
type R1<0> = Transcript<{ Commit::OBLIG_MASK }, 0>;
```